

TECNICATURA UNIVERSITARIA EN CIBERSEGURIDAD

DESARROLLO DE SOFTWARE SEGURO

TP Grupal – Entrega N° 1

Dante Gabriel Balbuena Atar

Daniel Avila Varas

Profesor: Juan Pablo Villalba

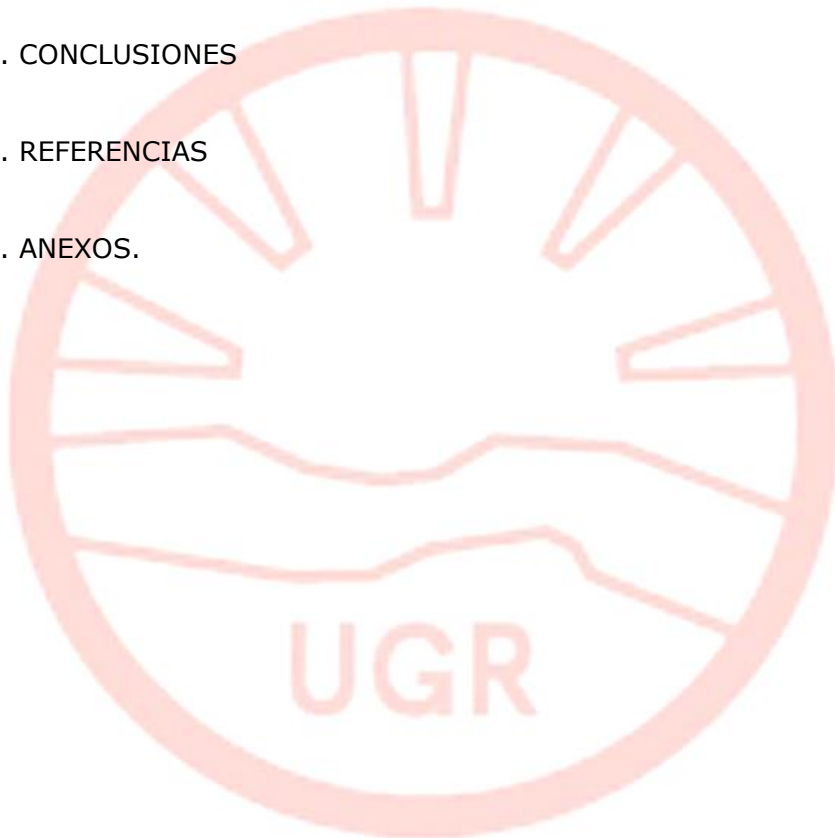
**Argentina, Tucumán, Tafí Viejo -
Chile, Santiago**

Fecha - 11/10/2025

Contenido

TP Grupal – Entrega N° 1

1. INTRODUCCIÓN
2. ANÁLISIS TEÓRICO
3. PARTE PRÁCTICA
4. CONCLUSIONES
5. REFERENCIAS
6. ANEXOS.



1. Introducción

En el desarrollo de software moderno, las Interfaces de Programación de Aplicaciones (APIs) son fundamentales para permitir la comunicación entre sistemas y el intercambio de datos mediante protocolos como HTTP o HTTPS. Su uso facilita la integración y escalabilidad de aplicaciones, pero también introduce riesgos de seguridad al exponer recursos sensibles.

La autenticación cumple un rol clave en la protección de las APIs, ya que permite verificar la identidad de los usuarios o sistemas que acceden a los servicios. Entre los principales métodos se encuentran la autenticación básica, por IP, mediante API Key y los esquemas avanzados como OAuth 2.0 y JWT (JSON Web Token), que ofrecen mayor control y trazabilidad.

En este trabajo se implementan y comparan dos frameworks de Python: **Flask** y **FastAPI**, evaluando su manejo de autenticación y seguridad. Además, se utiliza **OpenAPI** como estándar de documentación, lo que garantiza una definición clara e interactiva de los endpoints. Estos componentes sientan las bases para el desarrollo de **APIs seguras, bien documentadas y de fácil mantenimiento**.

2. Análisis teórico

2.1 Métodos de Autenticación en APIs

La autenticación en APIs constituye el primer nivel de defensa frente a accesos no autorizados. Su objetivo es verificar la identidad de los clientes o usuarios que intentan acceder a los recursos de un sistema. Existen diversos mecanismos de autenticación, que varían en complejidad, nivel de seguridad y aplicabilidad según el contexto de uso. A continuación, se describen los principales métodos abordados en esta entrega.

Método	Descripción	Ventajas	Limitaciones	Casos de uso recomendados
Autenticación Básica	Envía usuario y contraseña codificados en Base64 a través del encabezado HTTP. Generalmente se usa con HTTPS.	Simple de implementar y compatible con la mayoría de los frameworks.	Vulnerable si no se usa HTTPS; no permite control granular de acceso.	Pruebas iniciales, entornos controlados o APIs internas.
Autenticación por IP	Restringe el acceso a direcciones IP específicas, permitiendo	Muy segura en redes corporativas o entornos cerrados.	No es práctica con IPs dinámicas o servicios distribuidos;	Servidores internos, intranets o servicios restringidos.

	solicitudes solo desde fuentes confiables.		susceptible a spoofing.	
Autenticación por API Key	Usa una clave única generada para cada cliente que debe enviarse en cada solicitud.	Fácil de administrar y revocar; permite trazabilidad básica.	No cifra la información por sí misma; puede ser comprometida si se filtra.	Integraciones con terceros o microservicios controlados.
OAuth 2.0	Protocolo de autorización que delega permisos mediante “tokens” sin exponer credenciales del usuario.	Altamente seguro; permite control de permisos y expiración.	Implementación más compleja; requiere servidor de autorización.	Aplicaciones que acceden a recursos de terceros (p. ej., Google o Facebook APIs).
JWT (JSON Web Token)	Usa un token firmado digitalmente con información codificada (claims) para autenticación sin estado.	Rápido, escalable, y elimina la necesidad de sesiones en el servidor.	Si se filtra el token, el atacante tiene acceso hasta su expiración; requiere buena gestión de firmas.	APIs modernas, microservicios y autenticación distribuida.

2.2 Comparativa entre Flask y FastAPI

El desarrollo de APIs seguras depende del framework utilizado. En Python, **Flask** y **FastAPI** destacan por su flexibilidad, comunidad activa y soporte para entornos seguros, aunque difieren en arquitectura, rendimiento y nivel de automatización.

Aspecto	Flask	FastAPI
Arquitectura y rendimiento	Utiliza el estándar WSGI (modelo síncrono). Su procesamiento secuencial lo hace más simple, pero menos eficiente ante múltiples solicitudes simultáneas.	Implementa ASGI , permitiendo ejecución asíncrona y concurrente , lo que mejora la velocidad y escalabilidad en entornos de alta demanda.
Filosofía de diseño	Microframework minimalista: ofrece lo esencial, dejando al desarrollador libertad total para agregar extensiones o definir la estructura del proyecto.	Framework moderno orientado a la productividad y automatización , con múltiples componentes integrados (validación, seguridad, documentación).
Validación y manejo de datos	Requiere librerías externas como Marshmallow o Cerberus para validar entradas. Esto aumenta la	Integra Pydantic , que realiza validación automática y garantiza tipos de datos

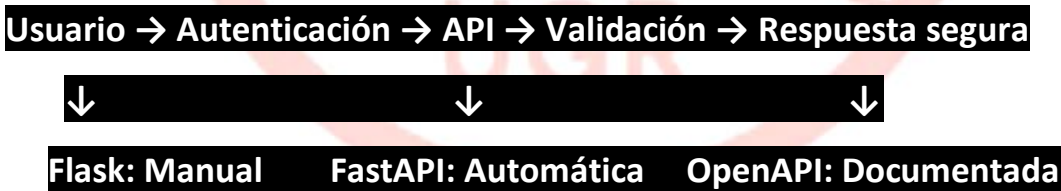
	flexibilidad, pero también la posibilidad de errores.	seguros mediante anotaciones (<i>type hints</i>).
Documentación OpenAPI	No genera documentación automática; requiere complementos como Flasgger o Flask-RESTX .	Genera automáticamente documentación interactiva con Swagger UI y ReDoc , conforme al estándar OpenAPI .
Seguridad	Ofrece flexibilidad mediante extensiones (por ejemplo, Flask-HTTPAuth o Flask-JWT-Extended), aunque depende de la correcta configuración del desarrollador.	Incorpora mecanismos nativos de autenticación (API Key, OAuth2, JWT) a través del módulo <code>fastapi.security</code> , reduciendo riesgos de configuración.
Escalabilidad y entorno de uso	Ideal para proyectos educativos, prototipos o sistemas pequeños/medianos , donde se valora el control manual.	Recomendado para entornos empresariales y microservicios , donde se prioriza la velocidad, concurrencia y cumplimiento de estándares modernos.

2.2.1 Síntesis comparativa

- **Flask:** destaca por su simplicidad y flexibilidad; es ideal para quienes buscan control total del código y una curva de aprendizaje gradual.
- **FastAPI:** sobresale en rendimiento, seguridad y productividad gracias a su diseño moderno y su documentación integrada.

En resumen, **Flask** es óptimo para proyectos formativos o personalizados, mientras que **FastAPI** representa la evolución natural hacia APIs seguras, escalables y profesionalmente mantenibles.

2.2.2 Resumen gráfico



2.3 Importancia de OpenAPI

OpenAPI es un estándar que permite **describir, documentar y validar APIs** de forma estructurada mediante formatos como **JSON o YAML**, garantizando claridad, interoperabilidad y seguridad en el desarrollo.

Su principal ventaja radica en la **documentación interactiva** que generan herramientas como **Swagger UI**, las cuales facilitan la exploración de endpoints, la verificación de autenticaciones y la detección temprana de errores o vulnerabilidades.

Además, OpenAPI contribuye a la **transparencia y trazabilidad**, permitiendo auditorías y control de accesos más eficientes.

En **FastAPI**, la documentación se genera automáticamente, mientras que en **Flask** requiere integración manual con librerías externas como *Flasgger* o *Flask-RESTX*.

3. Parte Práctica

3.1 Configuración del entorno (Debian GNU/Linux, Python, librerías)

El entorno se implementó sobre **Debian GNU/Linux 12 (Bookworm)**, en una máquina virtual configurada para el desarrollo seguro de APIs.

Se instalaron las dependencias necesarias para ejecutar **Flask** y **FastAPI**, garantizando un entorno limpio mediante el uso de un entorno virtual de Python.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv git curl
mkdir api_segura_tp && cd api_segura_tp
python3 -m venv venv
source venv/bin/activate
pip install flask fastapi uvicorn pydantic flask-httpauth
mkdir flask_api fastapi_api
```

(ver Figura 1 en Anexo 1)

El entorno quedó operativo para el desarrollo de las APIs, con todas las dependencias correctamente instaladas y aisladas.

3.2 Implementación en Flask

Se desarrolló una API básica en **Flask** que implementa **autenticación HTTP Basic Auth**, restringiendo el acceso a usuarios con credenciales válidas (admin:secret).

Archivo: flask_api/app.py (revisar enlace a git en la sección 3.5.3)

(ver Figura 2 en Anexo 1)

Esta implementación permite comprender los principios de la **autenticación básica mediante encabezados HTTP** en APIs REST.

3.3 Implementación en FastAPI

Se configuró una API equivalente utilizando **FastAPI**, aprovechando la clase HTTPBasic del módulo fastapi.security.

La ruta /api/protegida valida las credenciales antes de retornar una respuesta.

Archivo: fastapi_api/main.py (revisar enlace a git en la sección 3.5.3)

(ver Figura 3 en Anexo 1)

El servidor se ejecutó con el siguiente comando:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

(ver Figura 4 y 5 en Anexo 1)

3.4 Documentación OpenAPI

FastAPI genera automáticamente la documentación conforme al estándar **OpenAPI**, accesible desde la URL:

<http://localhost:8000/docs>

La interfaz **Swagger UI** muestra las rutas disponibles, los parámetros de autenticación y las respuestas esperadas.

En **Flask**, la documentación se podría incorporar con extensiones como flasgger o flask-swagger-ui, aunque en esta primera entrega se priorizó FastAPI por su soporte nativo.

3.5 Resultados de pruebas

3.5.1 Funcionalidad

Se realizaron pruebas de autenticación sobre ambas APIs mediante **HTTPIe** y la interfaz **Swagger UI**.

Las solicitudes autenticadas (admin:secret) obtuvieron acceso exitoso, mientras que las peticiones sin credenciales fueron rechazadas con código **401 Unauthorized**.

Ejemplo de prueba con HTTPIe:

`http -a admin:secret GET http://localhost:8000/api/protegida`

Respuesta:

```
{  
  "mensaje": "Acceso concedido a admin!"  
}
```

3.5.2 Documentación

La interfaz **Swagger UI** mostró correctamente las rutas y parámetros de autenticación, permitiendo la validación visual de las respuestas.

3.5.3 Repositorio

El código fuente del proyecto se encuentra alojado en GitHub:

👉 https://github.com/Dante2617012022/api_segura_tp

4. Conclusiones

La implementación práctica de este trabajo permitió no solo comprender los fundamentos teóricos de la autenticación y la documentación de APIs, sino también aplicarlos en un entorno técnico real mediante Flask, FastAPI y OpenAPI. Este proceso integró análisis, configuración y desarrollo seguro, fortaleciendo nuestras competencias tanto en programación como en ciberseguridad aplicada.

El hecho de documentar y subir el proyecto a GitHub constituye una evidencia tangible de nuestro progreso, permitiéndonos construir una base profesional verificable y alineada con las demandas del sector tecnológico actual. Estas experiencias nos preparan para aportar valor a empresas que priorizan la seguridad, la trazabilidad y las buenas prácticas de desarrollo.

Agradecemos especialmente el enfoque metodológico brindado, que fomenta el aprendizaje activo y el desarrollo de habilidades prácticas, esenciales para nuestra futura trayectoria profesional en el ámbito de la Ciberseguridad.

5. Referencias

Hardt, D. (2012). *The OAuth 2.0 Authorization Framework (RFC 6749)*. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc6749>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc7519>

Maina, T. (2023). *FastAPI Documentation*. Recuperado de <https://fastapi.tiangolo.com/>

Open Web Application Security Project (OWASP). (2023). *OWASP API Security Top 10*. Recuperado de <https://owasp.org/www-project-api-security-top-10/>

Pallets Projects. (2023). *Flask Documentation (Version 2.x)*. Recuperado de <https://flask.palletsprojects.com/>

Bhatti, R., Ghafoor, A., Bertino, E., & Joshi, J. (2005). *X-GTRBAC: An XML-Based Policy Specification Framework and Architecture for Enterprise-Wide Access Control*. *ACM Transactions on Information and System Security*, 8(2), 187–227. <https://doi.org/10.1145/1065545.1065547>

6. Anexos.

```
(venv) root@vbox:/home/fase2/api_segura_tp/fastapi_api# tree /home/fase2/api_segura_tp -L 2
/home/fase2/api_segura_tp
├── fastapi_api
│   ├── main.py
│   └── __pycache__
├── flask_api
│   └── app.py
└── venv
    ├── bin
    ├── include
    ├── lib
    ├── lib64 -> lib
    └── pyvenv.cfg

9 directories, 3 files
(venv) root@vbox:/home/fase2/api_segura_tp/fastapi_api#
```

Figura 1. La imagen muestra el entorno virtual (venv) activo en Debian y la estructura de carpetas creadas para el desarrollo: flask_api y fastapi_api. Esta organización facilita mantener separadas ambas implementaciones dentro del proyecto.

```

flask_api
├── app.py
└── venv
    ├── bin
    ├── include
    ├── lib
    ├── lib64 -> lib
    └── pyvenv.cfg

8 directories, 2 files
(venv) root@vbox:/home/fase2/api_segura_tp/flask_api# cd /home/fase2/api_segura_
tp/flask_api
(venv) root@vbox:/home/fase2/api_segura_tp/flask_api# ls
app.py
(venv) root@vbox:/home/fase2/api_segura_tp/flask_api# python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit

```

Figura 2. Ejecución de la API Flask.

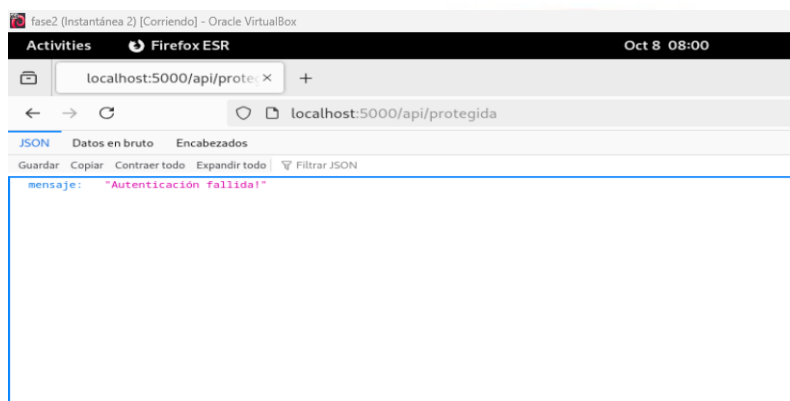


Figura 3. respuesta autenticada.

```

fase2@vbox: ~
(venv) root@vbox:/home/fase2/api_segura_tp/flask_api# python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
127.0.0.1 - - [08/Oct/2025 07:59:46] "GET /api/prottegida HTTP/1.1" 401 -
127.0.0.1 - - [08/Oct/2025 07:59:47] "GET /favicon.ico HTTP/1.1" 404 -
^C(venv) root@vbox:/home/fase2/api_segura_tp/flask_api# cd /home/fase2/api_segura_
tp/fastapi_api
(venv) root@vbox:/home/fase2/api_segura_tp/fastapi_api# nano main.py
(venv) root@vbox:/home/fase2/api_segura_tp/fastapi_api# uvicorn main:app --reloa
d --host 0.0.0.0 --port 8000
INFO: Will watch for changes in these directories: ['/home/fase2/api_segura_
tp/fastapi_api']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [39329] using StatReload
INFO: Started server process [39331]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Figura 4. Ejecución de la API FastAPI.



Figura 5. Respuesta autenticada.

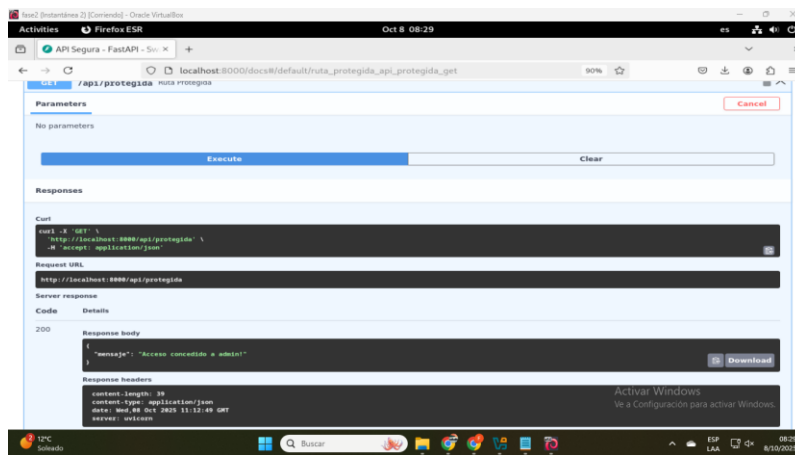


Figura 6. Vista de Swagger UI mostrando la ruta /api/protégida con respuesta “Acceso concedido a admin!”.